

Universidad Tecnológica Nacional (UTN)

- **Carrera:** Tecnicatura Universitaria en Programación
- **Materia:** Programación 2
- **Proyecto:** Sistema de Gestión de Pedidos - "Food Store"
- **Estudiantes:**
 - Lucia Ortiz (**Comisión:** COMI-9)
 - Juan Nahuel Montero (**Comisión:** COMI-9)
 - Hugo Insaurralde (**Comisión:** COMI-7)
- **Fecha de Entrega:** 22/06/2026

1. Introducción	[3]
2. Marco Teórico	[3]
3. Decisiones Técnicas y Arquitectura	[4]
4. Dificultades y Soluciones	[6]
5. Conclusión	[7]
6. Bibliografía	[9]

INTRODUCCIÓN

En el marco del Trabajo Final Integrador (TFI) para las cátedras de Programación 2. se presenta el sistema "Food Store". Este proyecto surge como una solución integral para la gestión de pedidos de un negocio gastronómico, permitiendo administrar de forma eficiente el inventario de productos, la organización de categorías, el registro de usuarios y el procesamiento transaccional de ventas. A través de este sistema, se busca no solo cumplir con los requerimientos funcionales de un CRUD (Alta, Baja, Modificación y Consulta), sino también aplicar estándares profesionales de arquitectura de software, tales como la separación en capas (Entities, Service y UI) y el cumplimiento de las propiedades ACID en la capa de persistencia.

En la dimensión de Programación, el foco se ha puesto en la robustez del código, utilizando una jerarquía de herencia basada en una clase abstracta Base, la implementación de interfaces para contratos de comportamiento y el manejo de excepciones personalizadas para el control de errores de negocio.

MARCO TEÓRICO

El proyecto se fundamenta en la Programación Orientada a Objetos (POO) utilizando el lenguaje Java . Se han aplicado los cuatro pilares fundamentales:

- **Abstracción:** Implementada a través de la clase Base y la interfaz Calculable.
- **Herencia:** Todas las entidades extienden de Base para compartir atributos de auditoría e identidad.
- **Encapsulamiento:** Uso de modificadores private y métodos Getters/Setters para proteger el estado de los objetos.
- **Polimorfismo:** Utilizado en la sobrescritura del método toString() y en la implementación de la interfaz Calculable.

Además, se aplicó la gestión de Colecciones Dinámicas (ArrayList) para el manejo de datos en memoria y una jerarquía de Excepciones Propias para garantizar la robustez ante errores de validación o entidades no encontradas.

DECISIONES TÉCNICAS Y ARQUITECTURA

Organización de Paquetes

El sistema se dividió en una arquitectura de capas para asegurar una alta cohesión y un bajo acoplamiento:

- **Main:** Contiene la clase MainConsola, encargada exclusivamente de la interacción con el usuario y el flujo de los menús.
- **Entities:** Contiene el modelo de dominio (Usuario, Pedido, Producto, etc.).
- **Service:** Capa de lógica donde residen los métodos CRUD y la gestión de las colecciones.
- **Enums:** Definición de tipos constantes para estados y roles.
- **Exceptions:** Clases personalizadas para el manejo controlado de errores.

Relaciones entre Objetos

- **Composición:** Se aplicó entre Pedido y DetallePedido. Los detalles se crean dentro del pedido, garantizando que su ciclo de vida dependa totalmente de este.
- **Agregación:** Entre Producto y Categoría, permitiendo que los productos existan independientemente de su clasificación.
- **Asociación Bidireccional:** Entre Usuario y Pedido, facilitando la trazabilidad de las compras desde ambos extremos.

Estructuras de Datos Dinámicas en Memoria

Al carecer el proyecto de un motor de bases de datos relacionales persistente, se implementó una estrategia de almacenamiento volátil basada en el Java Collections Framework. Se optó específicamente por la interfaz List provista mediante la implementación concreta de ArrayList. Esta estructura de datos lineal indexada ofrece un rendimiento óptimo de tiempo constante $O(1)$ para operaciones de lectura y acceso directo, permitiendo una gestión dinámica del tamaño de las colecciones a medida que se registran transacciones, superando las limitaciones operativas de los arreglos estáticos bidimensionales.

Modelo del Dominio e Interpretación UML

El diagrama de clases representa el mapa arquitectónico de las entidades que componen el negocio de Food Store. El diseño arquitectónico respeta y expande la estructura sugerida por la cátedra para garantizar un flujo lógico de la información y la consistencia de los datos.

- **Relación Uno a Uno Unidireccional Efectiva (1 a 1):** Como requerimiento de diseño riguroso, se incorporó una asociación lineal excluyente entre Pedido y Factura. Cada orden generada en el sistema da origen de forma automática a una única instancia de comprobante fiscal, asegurando el cumplimiento de las normativas de modelado avanzado sin sobrecargar la estructura de colecciones.

CAPTURAS

```

--- GESTION DE PRODUCTOS ---
1. Listar
2. Crear
3. Editar
4. Eliminar
0. Volver al Menu Principal
Seleccione: 1
--- LISTADO DE PRODUCTOS ---
ID: 1 | Nombre: Java cola | Precio: $1500.0 | Stock: 5 | Categoria: Bebidas sin alcohol
ID: 2 | Nombre: Tira de asado | Precio: $11000.0 | Stock: 10 | Categoria: Plato principal
ID: 3 | Nombre: Papas Fritas | Precio: $4500.0 | Stock: 10 | Categoria: Acompañamientos/guarniciones

```

```

--- GESTION DE PEDIDOS ---
1. Listar Pedidos
2. Iniciar Nuevo Pedido
3. Agregar Producto a un Pedido
4. Cancelar/Eliminar Pedido
0. Volver al Menu Principal
Seleccione: 2
--- LISTADO DE USUARIOS ---
No hay usuarios cargados.
Ingrese el ID del usuario para el pedido: 1
Error: No se encontro un usuario activo con el ID 1

```

```

--- GESTION DE PEDIDOS ---
1. Listar Pedidos
2. Iniciar Nuevo Pedido
3. Agregar Producto a un Pedido
4. Cancelar/Eliminar Pedido
0. Volver al Menu Principal
Seleccione: 1
--- LISTADO DE PEDIDOS ---
Pedido ID: 1 | Fecha: 2026-06-22 | Usuario: Charly Cimino | Estado: PENDIENTE | Total: $34000.0 | Factura: 0001-39250

```

DIFICULTADES Y SOLUCIONES

1. Dificultad: Implementación de la Composición Estricta en Pedidos.

- *Problema:* Inicialmente, existía la duda de si los objetos DetallePedido debían crearse en el Main o dentro del Pedido. Al crearlos fuera, se rompía la regla de Composición (Unidad 5), permitiendo que existieran detalles "huérfanos".
- *Solución:* Se encapsuló la instanciación de los detalles dentro del método addDetallePedido en la clase Pedido. De esta manera, el pedido es el único responsable de crear sus detalles, garantizando que el ciclo de vida de estos dependa totalmente del objeto padre.

2. Dificultad: Sincronización del Subtotal en el Detalle del Pedido.

- *Problema:* Al modificar la cantidad de un producto en un detalle ya existente mediante un Setter, el atributo subtotal quedaba desactualizado con el valor de la cantidad anterior.
- *Solución:* Se implementó un "Smart Setter" (Unidad 4). El método setCantidad no solo actualiza el valor del atributo, sino que invoca automáticamente al método privado calcularSubtotal(), asegurando la integridad del dato en todo momento.

3. Dificultad: Gestión de la Identidad en la Clase Abstracta Base.

- *Problema:* Al heredar de la clase Base (Unidad 7), surgieron errores de compilación porque los constructores de las clases hijas no inicializaban correctamente los atributos de auditoría (id, eliminado, createdAt).
- *Solución:* Se aplicó la "Regla de Oro de la Inicialización", utilizando la instrucción super() en la primera línea de cada constructor hijo para forzar el nacimiento correcto de la superclase antes que la subclase.

CONCLUSIÓN

El desarrollo integral del sistema Food Store ha permitido consolidar los conocimientos adquiridos a lo largo de las ocho unidades de la materia, logrando una sinergia entre el diseño de software orientado a objetos y la gestión de datos relacionales.

Desde la perspectiva de la Programación, la implementación de una arquitectura basada en capas (Entities, Service y UI) resultó fundamental para garantizar una alta cohesión y un bajo acoplamiento. El uso de la clase abstracta Base y la interfaz Calculable permitió demostrar cómo los contratos de comportamiento y la herencia estructuran sistemas escalables. Asimismo, la gestión de Excepciones Propias y el uso de Colecciones Dinámicas dotaron al programa de la robustez necesaria para procesar transacciones en memoria sin comprometer la integridad del modelo ante errores de usuario.

BIBLIOGRAFÍA

- Oracle Java Documentation: <https://docs.oracle.com/en/java/>
- Material de Cátedra UTN - Unidades 1 a 8 (Programación 2).
- Repositorio del Proyecto:
https://github.com/HugoInsaurralde/Tpi_Programacion2/tree/main
- Link video presentación: <https://www.youtube.com/watch?v=K46FJYgmlPM>